Supplemental Material for Second-level termination of the PPT entanglement-cost hierarchy for two qutrits

Wei Song* and Xiao-Lan Zong†
School of Physics and Materials Engineering, Hefei Normal University, Hefei 230601, China

Ming Yang
School of Physics, Anhui University, Hefei, 230601, China and
Leibniz International Joint Research Center of Materials Sciences of Anhui Province, Anhui University, Hefei, 230601, China

This guide is the companion documentation for the programs and data in the supplementary material of the paper; it describes the rational certificate data and explains how to perform exact verification. Sections III–V of the main text give the mathematical reduction, the finite-certificate argument, and the analytic proof for the boundary region, and Appendix A describes the verification steps. The running commands, environment requirements, and the complete file list are given in `README.TXT`.

S1. CERTIFICATE DOMAINS

We continue to use the notation of the main text. After spectral decomposition and simplification, we have

$$C = C_0 + E, \qquad C_0 = |r\rangle\langle r|, \qquad r = x|00\rangle + y|11\rangle + t|22\rangle, \tag{S1}$$

where $1 \geq x \geq y \geq t > 0$ and $2 < c = x^2 + y^2 + t^2 \leq 2 + t^2$. We take $X = sC_0 + \alpha E$, where $s = (1+\beta)/2$. Under the dual feasibility conditions of Lemma 2 of the main text, the certificate data can be used to verify that the matrix

$$P_X = I + H^\Gamma + (1-2\alpha)E^\Gamma - \beta C_0^\Gamma \tag{S2}$$

is positive semidefinite throughout the entire region listed in Table SI.

TABLE SI. Finite certificate files and their coverage ranges. The last column gives the constant m in the verified lower bound $P_X \succeq mI$, which holds for all parameters in the corresponding region. All intervals include their endpoints.

File	Range of t	Number of certificate groups	m
`data/dyadic.json`	$[1/640, 1/10]$	6	1/100000
`data/middle.json`	$[1/10, 13/16]$	119	1/5000
`data/upper.json`	$[13/16, 1]$	1	1/100

The dyadic region consists of six rectangles in the (x, t) plane:

$$[1 - T_k^2/2, 1] \times [T_k/2, T_k], \qquad T_k = \frac{1}{10\,2^k}, \quad k = 0, \ldots, 5. \tag{S3}$$

The file for the middle region gives a binary partition of the rectangle $[13/16, 1] \times [1/10, 13/16]$, splitting each time at the exact midpoint of the chosen interval. The partition tree has 243 nodes in total, of which 119 leaf nodes are verified by certificates, and the other three leaf nodes are excluded because they satisfy $2u^2 + b^2 < 2$. Here, u, b are the upper bounds of x, t in the rectangle corresponding to that leaf node, respectively. The file for the upper region covers the cube $[13/16, 1]^3$, with certificate parameters $\alpha = s = 2/3$.

For both the middle region and the dyadic region, $\alpha = 1/2$. For a record corresponding to the interval $t \in [a, b]$, the stored parameter is $\beta = 1 - (a+b)/2$. The verification program checks not only positive semidefiniteness but also the norm conditions and the various parameter constraints. Section V of the main text gives the analytic proof for $0 < t \leq 1/640$, including the endpoint $t = 1/640$ shared with the finite-certificate region. The case $\|C\| \leq 2$ is handled directly in Section III of the main text.

* wsong1@mail.ustc.edu.cn
† zxl@hfnu.edu.cn

S2. RATIONAL DATA AND COEFFICIENT BOUNDS

The certificate files store integer arrays and strings representing rational numbers. The program reconstructs each map from integer Gram factors, without using precomputed floating-point eigenvalues. If the numerator array of the Gram factor is G and the denominator is d, then the corresponding Choi block is

$$J_{\mathrm{block}} = \frac{GG^{\mathsf{T}}}{d^2}. \tag{S4}$$

For the middle and upper regions, $d = 10^8$; for the dyadic region, $d = 10^{10}$. In the scalar certificates for the middle and dyadic regions, the Gram factors also use their respective denominators.

The indices of the Choi matrix are flattened according to $(p, b) \mapsto 9p + b$, and the corresponding map is defined as

$$\Phi(A)_{bd} = \sum_{p,q} A_{pq} J_{(p,b),(q,d)}. \tag{S5}$$

All maps have output dimension nine. Except that the input dimension of S is three, the input dimensions of the remaining maps are also nine. The key names `AH,AHT,AE,AET,AR,ART` in the files correspond respectively to $A_H, A_{HT}, A_E, A_{ET}, A_R, A_{RT}$ in the main text, where the input transposes involved are all full transposes. Together with U, Q, B, V, S, each certificate group contains eleven maps in total.

Matrix residuals are expanded in the order of the seven monomials $1, x^2, y^2, t^2, xy, xt, yt$. The scalar polynomial certificates use $z_0 = (1, x, y, t)^{\mathsf{T}}$ and the nonnegativity constraints listed in Section IV of the main text. The program exactly reconstructs the Gram terms and scalar weights therein, and computes the polynomial coefficient errors. For the file of the upper region, dividing the integer rows in `lower_polynomial_rows` by 10000 gives the corresponding polynomial coefficients, with the order still consistent with the above seven monomials.

The rational data approximately satisfy the identities used in the construction, and the corresponding errors are also retained in the verification. Following the notation of the paper, the program recomputes the matrix coefficient error bound δ, the row-sum upper bound η for the absolute values of off-diagonal entries, and the scalar polynomial error $\varepsilon_{\mathrm{pol}}$, and then verifies for the middle and dyadic regions

$$P_X \succeq (\mu - \varepsilon_{\mathrm{pol}} - \eta - \delta) I. \tag{S6}$$

For the upper region, it uses the polynomial lower bounds obtained by comparing coefficients term by term in Section IV of the main text. All comparisons are carried out using exact rational arithmetic.

S3. COMPLEX INPUTS AND THE FULL AUDIT

These maps are complex-linear and preserve Hermiticity. For the coefficient error map $\mathcal{R}$,

$$\mathcal{R}(E_{ji}) = \mathcal{R}(E_{ij})^{\dagger}. \tag{S7}$$

Therefore, the values of the map on the 45 matrix units with $i \le j$ determine the error on all Hermitian inputs. When computing the row-sum upper bound, the pair of adjoint terms corresponding to each off-diagonal matrix unit is included. This is necessary for handling general complex inputs.

For the 126 certificate groups and three matrix variables, routine verification computes map values on $126 \times 3 \times 45 = 17010$ matrix units in total. The complete audit also directly computes the values on the remaining $126 \times 3 \times 36 = 13608$ matrix units, checks the adjoint identities, and compares the nine cumulative row sums one by one. Therefore, the complete audit directly computes map values on 30618 matrix units in total. The coefficient error bounds given by direct computation and by adjoint-pairing computation are exactly the same. The above counts refer only to the finite-certificate audit and do not include the separately performed index checks and intentionally introduced-error tests.

S4. PROGRAMS AND REPRODUCTION

`run_verification.py` runs the finite-certificate verification program and the boundary auxiliary verification module sequentially, and stops if either module fails. The finite-certificate verification program first checks the hash values of the seven input files listed in `SHA256SUMS.json`, and then checks the maps, scalar certificates, norm conditions,

and parameter-domain coverage. The hash values are used to confirm that the input files are identical to the released version, while the certificate inequalities are verified by rational computation.

`verify_boundary.py` checks the scalar rational inequalities in Section V of the main text, the partial transpose on all 81 matrix units, the symmetric projector identities, and the index permutations used in the Frobenius norm estimates. The program also deliberately omits a transpose or a leaf node used to cover the parameter domain, and checks whether the corresponding test reports an error. These changes are made only in memory and do not modify the original data files.

Running the programs requires only Python 3.9 or higher and its standard library. In the extracted supplementary-material directory, run

```
python3 run_verification.py --audit-all-units
```

to reproduce the complete audit of the released version. Since assertions participate in verification, do not enable Python optimized mode when running. Without the audit option, the program uses adjoint-pairing computation by default. Commands under Windows, expected output, and handling of common issues are given in the README file.

The attached `verification.log` stores all output of one complete audit, which used files fully extracted in a clean directory. The programs have been tested in a CPython 3.12.14 environment on Linux. The three certificate data files are the same as those attached to the previous manuscript version, and reproducing these computations does not require re-searching certificates or performing SDP optimization.

The inequalities in the finite certificates are verified by exact computation. The matrix conclusions in the general case are proved in the paper, including the analytic argument for the boundary region and the treatment of singular matrix blocks. The program verifies the given certificates and auxiliary identities, but does not constitute a formal verification of the entire written proof.